

Andromeda Rust 2026 Architecture and Engineering Guidance

Download copies: [Markdown](#) · [PDF](#)

This report treats **Andromeda** as a systems-grade Rust initiative and focuses on the full engineering philosophy around it: architecture, repository structure, coding standards, testing, hardware-conscious implementation, and operational governance. No Andromeda-specific internal repository or design document was available in connected sources during this research pass, so the recommendations below are intentionally written as a high-confidence 2026 baseline that you can specialize once the project class is fixed more precisely.

Executive position

As of **2026-05-07**, the current stable release of Rust is **1.95.0**, published on **2026-04-16**. The **Rust 2024** edition stabilized with **Rust 1.85.0** on **2025-02-20**, and Rust continues to ship on its established six-week release cadence. For a project that wants to be meaningfully modern in 2026, the correct baseline is therefore not “Rust in general,” but **edition-2024, stable-toolchain, workspace-governed Rust**. ¹

The larger ecosystem signal points in the same direction. The 2024 and 2025 State of Rust survey reports show increasing non-trivial organizational use of Rust, continued hiring interest, and an ongoing rise in daily professional Rust usage. At the same time, **resource usage and compile times remain major productivity pain points**, and debugging also remains a significant concern. That combination matters for Andromeda: the project will gain more by standardizing repository architecture, dependency policy, tests, and profiling discipline than by chasing isolated language tricks. ²

Architecture for Andromeda

The strongest architectural default for Andromeda is a **workspace-first, domain-oriented design**. Cargo workspaces provide the right control plane for a professional codebase: common commands across members, a shared `Cargo.lock`, a shared output directory, inherited dependency policy, and inherited lint policy via `workspace.lints`. In practice, that makes the workspace root the correct place for build profiles, lint inheritance, policy metadata, and CI conventions. ³

For the application shape itself, the most defensible 2026 model is **DDD-inspired layering**: define bounded contexts first, then keep aggregates and domain rules inside crates that are independent from transport, persistence, and runtime choices. Microsoft’s current DDD guidance still emphasizes bounded contexts, aggregates, domain services, and domain events as the mechanism for producing cohesive service and subsystem boundaries; it also explicitly recommends that service boundaries should not be smaller than an aggregate and not larger than a bounded context. That maps unusually well to Rust workspaces, where crate boundaries can encode those same design boundaries precisely. ⁴

For Andromeda, that yields a concrete recommendation: keep **domain crates** pure and framework-light; keep **application crates** focused on use-case orchestration; keep **infrastructure crates** as adapters for databases, filesystems, network protocols, brokers, OS services, or hardware; and keep **binary crates** responsible only for composition, startup, observability, and shutdown. That structure is not a literal rule from Rust documentation; it is the synthesis that best aligns Cargo workspace governance with tactical DDD boundaries and with Rust's explicitness around ownership, traits, and dependency surfaces. ⁵

A practical repository skeleton for Andromeda is:

```
andromeda/
|- Cargo.toml                # virtual workspace root
|- rust-toolchain.toml
|- .cargo/
|   |- config.toml
|   `-- nextest.toml
|- supply-chain/
|   |- audits.toml
|   `-- config.toml
|- crates/
|   |- andromeda-domain/
|   |- andromeda-application/
|   |- andromeda-contracts/
|   |- andromeda-infra-db/
|   |- andromeda-infra-net/
|   |- andromeda-platform/
|   |- andromeda-runtime/
|   |- andromeda-cli/
|   |- andromeda-service/
|   `-- andromeda-embedded/    # optional, only if firmware/no_std exists
|- benches/
|- docs/
|- examples/
`- scripts/
```

If Andromeda spans multiple deployment modes, do not force them into one crate graph indiscriminately. The Embedded Rust documentation explicitly distinguishes hosted and `no_std` environments, while the Linux kernel Rust documentation makes clear that kernel Rust remains a specialized context with its own constraints. That means embedded, firmware, kernel-adjacent, CLI, and service slices should be **separated structurally**, even if they share domain semantics. ⁶

Rust coding standard

The code standard should begin with **edition and resolver policy**. In Rust 2024, `edition = "2024"` implies `resolver = "3"`, which turns on Rust-version-aware dependency resolution by default. Cargo's `rust-version` field also has real operational value in 2026: it improves diagnostics, influences dependency selection, and defines a support contract that CI can verify. In other words, a serious Rust project should treat `edition`, `resolver`, and `rust-version` as architectural settings, not incidental manifest metadata. ⁷

At the workspace level, enforce formatting and lint inheritance centrally. Cargo workspaces support inherited lint configuration, and the Linux kernel's Rust coding guidance still recommends idiomatic `rustfmt` as the baseline formatting authority. The Rust Performance Book also recommends Clippy broadly, including for performance-related code smells. For Andromeda, the right default is workspace-level lint inheritance, `cargo fmt --check`, and `cargo clippy` in CI, with stricter denies in foundational crates than in experimental edge crates. ⁸

For public API design, the Rust API Guidelines remain the best primary source. They strongly reinforce practices that should become mandatory in Andromeda: private fields for future-proofing, sealed traits when downstream implementations must remain closed, newtypes for semantic distinctions, builders for complex construction, additive features only, and meaningful, well-behaved error types. Those guidelines are especially valuable because they align API stability with long-term changeability, which is essential if Andromeda will grow into a multi-crate platform. ⁹

Error strategy should be split by layer. For reusable libraries and domain crates, prefer typed errors using `thiserror`; the crate's own documentation explicitly notes that it derives `std::error::Error` implementations without leaking `thiserror` into the public API. For binaries, orchestration, tools, and glue code, `anyhow` remains a pragmatic choice because it adds context cleanly and preserves causal chains for debugging. That is the most maintainable compromise between library-quality semantics and application-level ergonomics. ¹⁰

Documentation should be treated as executable engineering output. The rustdoc book recommends crate-level front pages, public documentation for public items, examples that users can copy and run, and explicit sections such as `# Panics`. Rustdoc also supports documentation tests, which means examples can become part of the build contract rather than narrative-only text. For Andromeda, the professional rule should be simple: public APIs are not complete until they have docs, examples, and failure-mode notes. ¹¹

Unsafe code needs its own governance policy. The Rust 2024 edition made several unsafety boundaries more explicit: `unsafe_op_in_unsafe_fn` now warns by default, certain attributes such as `no_mangle`, `export_name`, and `link_section` must now be marked `unsafe`, and `extern` blocks must be marked `unsafe`. The Rust Reference also explicitly enumerates these operations as part of the language's unsafety model. For Andromeda, the correct 2026 posture is **unsafe denied by default**, with isolated exceptions, explicit `SAFETY:` comments, and separate review pressure on those crates. ¹²

Testing, TDD, and verification

The most useful translation of TDD into Rust in 2026 is not ideology; it is **design pressure**. Martin Fowler's current TDD summary still defines the core cycle as **red, green, refactor**, and his test pyramid guidance still argues for many more low-level unit tests than broad-stack UI tests. Microsoft's current testing guidance aligns with that direction by emphasizing testability early, reliability, and avoiding brittle high-cost tests. For Andromeda, that means TDD is most valuable in domain logic, protocols, parsers, critical invariants, and bug-fix reproduction, while the overall test portfolio should remain pyramid-shaped rather than end-to-end-heavy. ¹³

For the day-to-day test runner, **cargo-nextest** is a strong 2026 default. Its documentation shows first-class support for retries, explicit flaky-test handling, per-test overrides, thread requirements, and repository-level configuration. That is exactly the sort of operational maturity Andromeda should want

in CI. The important caveat is governance: retries are useful for quarantining known nondeterminism, not for normalizing poor tests. ¹⁴

The verification stack should go beyond unit tests. `cargo-hack` is specifically built to exercise workspace members, features, and version ranges that normal Cargo workflows do not cover well; `cargo-msrv` is built to find and verify the declared MSRV; Miri detects undefined behavior in unsafe code and can run test suites; Loom explores concurrent interleavings for Rust concurrency primitives; Kani uses model checking to analyze Rust code for safety and correctness; and cargo-mutants uses mutation testing to detect test suites that execute code without truly checking behavior. Together, those tools give Andromeda a much stronger quality model than ordinary coverage numbers. ¹⁵

The practical implication is straightforward. Foundational crates should aim for fast unit tests, narrow integration tests, and doctests as part of the normal pull-request workflow. High-risk crates—especially any crate containing `unsafe`, lock-free logic, or protocol-critical state transitions—should also receive Miri, Loom, Kani, or mutation testing selectively. That is not because every crate needs maximal proof machinery; it is because the project should concentrate stronger evidence where the blast radius is largest. ¹⁶

Low-level, hardware, and performance engineering

Rust’s low-level value proposition in 2026 is real, but it must be handled deliberately. The Embedded Rust Book explicitly frames embedded work around cross-compilation, peripherals, interrupts, `no_std`, and host-versus-bare-metal constraints. The Embassy Book builds on that by making `async` await a first-class embedded option, with statically allocated tasks, timer integration, and HAL support across major embedded families. Probe-rs, in turn, provides flashing, debugging, RTT/defmt logging, and integration with mainstream editors and debug adapters. If Andromeda has a firmware or board-facing track, that track should be a **separate architecture lane**, not an accidental extension of the main service codebase. ¹⁷

Kernel-adjacent work needs even more caution. Linux kernel Rust support is real and documented, but the kernel documentation still states that Rust support is aimed primarily at kernel developers and maintainers and is still experimental for several configurations, not a blanket “use this for everything” recommendation. If Andromeda needs kernel modules, drivers, or privileged runtime components, treat that as a specialized subprogram with its own toolchains, safety cases, and review standards. Do not use kernel Rust merely because it is available. ¹⁸

Performance work should stay measurement-driven. The Rust Performance Book emphasizes profiling before optimization, prioritizing algorithm and data-structure changes over low-level micro-tuning, and using tools such as `perf`, VTune, Cachegrind, DHAT, `samply`, and `flamegraph` depending on the platform. It also calls out practical necessities such as release builds, line-table debug info, and frame pointers for high-quality profiles. Criterion.rs complements that by providing statistically driven benchmarking and regression detection, while Cargo’s own `--timings` report visualizes compilation bottlenecks and critical paths. In 2026, the modern Rust team is not the team that guesses fastest; it is the team that can prove where time and memory go. ¹⁹

For SIMD and extremely hardware-conscious code, stay conservative. Stable `std::arch` remains the proper gateway for architecture-specific intrinsics, with explicit CPU-feature detection and fallback implementations. By contrast, `std::simd` portable SIMD is still marked as a nightly-only experimental API in current documentation. That means Andromeda should treat portable SIMD as an optional experiment, not as the default production baseline. Likewise, `std::hint::black_box` is useful for

benchmarking realism, but the standard library explicitly says it is best-effort and must not be relied upon for correctness. ²⁰

FFI and build scripts should be narrowed aggressively. Cargo's build-script documentation is explicit that scripts run before builds, should write only into `OUT_DIR`, should emit precise `rerun-if-changed` and `rerun-if-env-changed` directives, and should be treated carefully because they affect build reproducibility and compile times. For Andromeda, the operational rule should be: isolate FFI and native build code into adapter crates, keep `build.rs` minimal, and never let platform glue expand into a hidden second build system. ²¹

Cookbooks and operational baseline

A mature Rust codebase should define its **canonical references** explicitly. The Rust documentation bookshelf is the central index for the Rust Book, the Reference, the standard library docs, and related official material. The Rust Cookbook remains a practical task-oriented collection aimed at common programming tasks and ecosystem patterns. Together with the Cargo Book, the Edition Guide, rustdoc guidance, and the Performance Book, they form a strong baseline reference set for Andromeda's onboarding and code review culture. ²²

Supply-chain and dependency governance should also be part of the baseline architecture, not bolted on later. RustSec provides the advisory database behind `cargo-audit`; cargo-deny can check advisories, licenses, banned crates, duplicate versions, and dependency sources; and cargo-vet is designed to help projects ensure that third-party dependencies have been audited by trusted parties, with in-repo policy and exemption tracking. For a project with Andromeda's ambitions, this should be standard CI policy from the beginning. ²³

If Andromeda includes service workloads, a pragmatic stack is available today: Tokio provides the runtime primitives, `tokio::select!`, channels, `JoinSet`, and documented graceful-shutdown patterns; `tracing` gives structured spans and event-based diagnostics designed specifically to work well in asynchronous systems; Axum supports typed state extraction and stateful router composition; and SQLx provides compile-time checked query macros plus an offline mode that can be validated in CI. The architectural rule is important, though: these are **edge and infrastructure choices**, not reasons to pollute the domain model. ²⁴

Recommended standard for Andromeda

Adopt the following default unless a concrete Andromeda requirement disproves it:

- Use **Rust 2024** on stable Rust, with an explicit `rust-version` policy and edition-aware dependency resolution. ²⁵
- Make the repository a **virtual Cargo workspace** with root-owned profiles, lint inheritance, and supply-chain metadata. ²⁶
- Structure the codebase around **bounded contexts and aggregates**, with pure domain crates and adapter crates around them. ⁴
- Enforce `rustfmt`, Clippy, public docs, doctests, and explicit failure-mode documentation as part of the definition of done. ²⁷
- Deny `unsafe` by default, isolate exceptions, and require explicit local safety justifications. ¹²
- Standardize on **Nextest + Criterion** for mainline test and benchmark execution, then add Miri, Loom, Kani, and cargo-mutants selectively to the highest-risk crates. ²⁸

- Add **cargo-hack** and **cargo-msrv** to CI so feature matrices and declared support policy are continuously verified. ²⁹
- Treat **cargo-audit**, **cargo-deny**, and **cargo-vet** as baseline supply-chain controls. ³⁰
- If the runtime is service-oriented, prefer **Tokio + tracing**, with Axum or Tower composition where appropriate and SQLx when compile-time checked SQL is genuinely beneficial. ³¹
- If the runtime is hardware-facing, split out an embedded slice built around **no_std** **discipline**, **Embassy**, and **probe-rs**, while keeping business rules outside firmware crates. ³²
- If extreme hardware specialization is required, reach for stable **std : : arch** only after profiling proves the need; do not make nightly-only **std : : simd** your baseline plan. ³³

The most important conclusion is this: **Rust is not modern in 2026 merely because the code compiles in Rust**. It becomes modern when the project uses Rust’s strengths end to end—workspace governance, explicit architecture, narrow unsafe boundaries, reproducible builds, strong documentation contracts, structured observability, and verification tools that match the risk profile of each subsystem. ³⁴

Open questions and limitations

Because no Andromeda-specific internal design source was available in connected documents, this report cannot yet specialize the baseline for one precise project class such as firmware, kernel/module work, service platform, desktop tooling, or high-frequency compute. If Andromeda clearly belongs to one of those classes, the next refinement should stop being multi-modal and turn this baseline into one concrete target architecture with crate-by-crate conventions, CI matrices, and runtime-specific standards.

-
- ¹ <https://blog.rust-lang.org/2026/04/16/Rust-1.95.0/>
<https://blog.rust-lang.org/2026/04/16/Rust-1.95.0/>
 - ² <https://blog.rust-lang.org/2025/02/13/2024-State-Of-Rust-Survey-results/>
<https://blog.rust-lang.org/2025/02/13/2024-State-Of-Rust-Survey-results/>
 - ³ ⁵ ²⁶ ³⁴ <https://doc.rust-lang.org/nightly/cargo/reference/workspaces.html>
<https://doc.rust-lang.org/nightly/cargo/reference/workspaces.html>
 - ⁴ <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis>
<https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis>
 - ⁶ ³² <https://docs.rust-embedded.org/book/intro/no-std.html>
<https://docs.rust-embedded.org/book/intro/no-std.html>
 - ⁷ ²⁵ <https://doc.rust-lang.org/stable/edition-guide/rust-2024/cargo-resolver.html>
<https://doc.rust-lang.org/stable/edition-guide/rust-2024/cargo-resolver.html>
 - ⁸ <https://doc.rust-lang.org/cargo/reference/workspaces.html>
<https://doc.rust-lang.org/cargo/reference/workspaces.html>
 - ⁹ <https://rust-lang.github.io/api-guidelines/checklist.html>
<https://rust-lang.github.io/api-guidelines/checklist.html>
 - ¹⁰ <https://docs.rs/thiserror/latest/thiserror/index.html>
<https://docs.rs/thiserror/latest/thiserror/index.html>
 - ¹¹ <https://doc.rust-lang.org/rustdoc/how-to-write-documentation.html>
<https://doc.rust-lang.org/rustdoc/how-to-write-documentation.html>

12 <https://doc.rust-lang.org/edition-guide/rust-2024/unsafe-op-in-unsafe-fn.html>
<https://doc.rust-lang.org/edition-guide/rust-2024/unsafe-op-in-unsafe-fn.html>

13 <https://martinfowler.com/bliki/TestDrivenDevelopment.html>
<https://martinfowler.com/bliki/TestDrivenDevelopment.html>

14 28 <https://nexte.st/docs/features/retries/>
<https://nexte.st/docs/features/retries/>

15 29 <https://github.com/taiki-e/cargo-hack>
<https://github.com/taiki-e/cargo-hack>

16 <https://github.com/rust-lang/miri>
<https://github.com/rust-lang/miri>

17 <https://docs.rust-embedded.org/book/>
<https://docs.rust-embedded.org/book/>

18 <https://docs.kernel.org/6.12/rust/index.html>
<https://docs.kernel.org/6.12/rust/index.html>

19 <https://nnethercote.github.io/perf-book/profiling.html>
<https://nnethercote.github.io/perf-book/profiling.html>

20 33 <https://doc.rust-lang.org/std/arch/index.html>
<https://doc.rust-lang.org/std/arch/index.html>

21 <https://doc.rust-lang.org/cargo/reference/build-scripts.html>
<https://doc.rust-lang.org/cargo/reference/build-scripts.html>

22 <https://doc.rust-lang.org/>
<https://doc.rust-lang.org/>

23 30 <https://rustsec.org/>
<https://rustsec.org/>

24 31 <https://tokio.rs/tokio/topics/shutdown>
<https://tokio.rs/tokio/topics/shutdown>

27 <https://docs.kernel.org/rust/coding-guidelines.html>
<https://docs.kernel.org/rust/coding-guidelines.html>